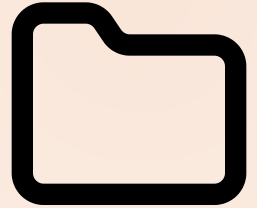


MODULE 06 · 22 MIN

Git Workflows for Safe AI Dev

Never let Claude push to main. Branch, commit atomically, PR — you stay the gate.



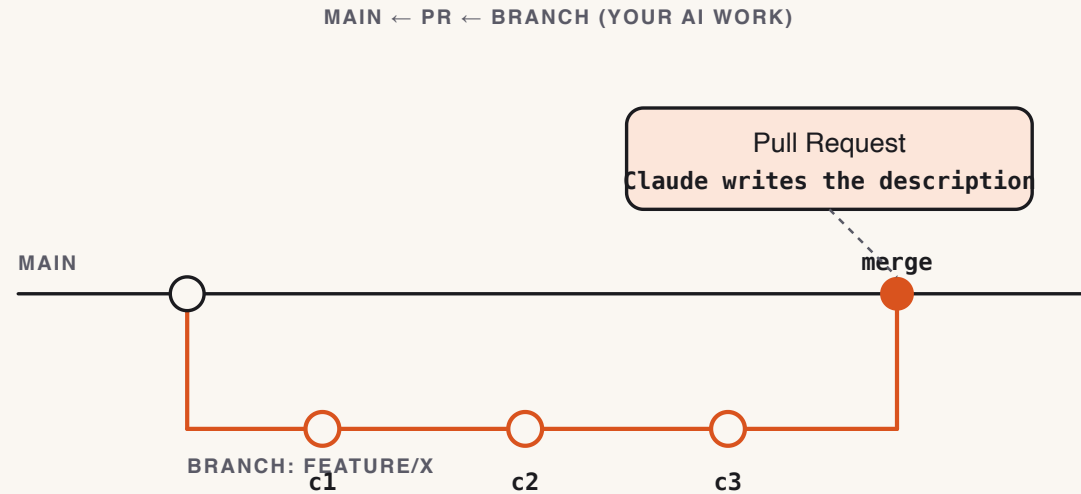
Theory · Safe git for AI code (4 min)

- **Branch first**, always: `<type>/<scope>-<summary>` → `feat/notes-api-search`.
- **Atomic commits** — one logical change each. Claude can split a dirty tree if you ask.
- **Conventional Commits**: `feat:` · `fix:` · `chore:` · `docs:` · `test:`.
- **PR description shape**: What changed · Why · How to test · Risk · Rollback.

Claude writes the commit messages and PR — but **from the actual diff**, never from your prompt.

Full reference: [skills/git-workflow/SKILL.md](#).

Branch → atomic commits → PR



Branch first · **atomic Conventional commits** · PR explains What · Why · Test · Risk · Rollback.

Reference · Bonus · @claude GitHub Action

`anthropics/claude-code-action` turns Claude into a **teammate in your repo**:

- Mention `@claude` in an issue or PR comment → it proposes a fix.
- Automated PR review on every push.
- Issue-to-PR flow: describe the bug, get a draft PR.

```
# .github/workflows/claude.yml (sketch)
uses: anthropics/claude-code-action@v1
with:
  anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
```

Stretch goal in the exercise — wire it on a throwaway repo.

Reference · Common mistakes

- One giant commit called `feat: stuff` — re-run the splitter.
- PR that says *what* but not *why* — reviewers reject it.
- Writing the PR from the prompt instead of the diff.
- Pushing to main (always branch first).

Live demo · Split commits, write the PR (5 min)

1. On the Module 5 tree (dirty): `git switch -c feat/notes-api-tests-and-fixes`. Then paste:

Group these staged changes into atomic commits using Conventional Commit subjects. Show the plan (files per commit + message) before committing.

2. Apply and commit. Then paste:

Write a PR description from the branch diff: What, Why, How to test, Risk, Rollback.

3. Review, edit, open a draft PR (or simulate).

Success signal: ≥ 3 atomic commits with Conventional subjects; PR explains *why*, not just *what*.

▶ Your turn · Branch → commits → PR (10 min)

Exercise: `exercises/part-06/README.md`

Take your Module 5 work onto a feature branch and ship a clean history:

- Create a feature branch; ask Claude to split into **≥ 3 atomic commits**.
- Generate a PR description from the **diff** (What · Why · How to test · Risk · Rollback).

Deliverables: `branch.txt` (name + `git log --oneline`) · `commits.md` · `pr.md`.

Success signal: a mergeable PR with sensible Conventional-Commit messages.

✓ Done & next (1 min)

Definition of done

- ☐ [] Feature branch holds all Module 5 work.
- ☐ [] ≥ 3 atomic commits with Conventional subjects.
- ☐ [] `pr.md` has all sections: Summary · Why · What changed · How to test · Risk · Rollback.

Next — code is safe in git. Now we feed Claude a *picture* and build a UI.

Module 7 — Multimodal: Screenshot to UI.